

Theory of Automata

CS-3005

Lecture 16

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Simplification of Context Free Grammars

- Substitution Rule
- Removing λ -productions
- Removing Unit Productions/Chain rules
- Removing Useless symbols
- Augmented a Grammar
- Removing Left Recursion
- Left Factoring

A Substitution Rule

$$S \rightarrow aB$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc$$

$$B \rightarrow aA$$

$$B \rightarrow b$$

Substitute
 $B \rightarrow b$

Equivalent
grammar

$$S \rightarrow aB \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$

$$S \rightarrow aB \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$

Substitute

$$B \rightarrow aA$$

$$S \rightarrow \cancel{aB} \mid ab \mid aaA$$

$$A \rightarrow aaA$$

$$A \rightarrow \cancel{abBc} \mid abbc \mid abaAc$$

Equivalent
grammar

In General

$$A \rightarrow xBz$$

$$B \rightarrow y_1$$

Substitute

$$B \rightarrow y_1$$

$$A \rightarrow xBz \mid xy_1z$$

equivalent
grammar

Context Free Grammars Killing λ -Productions

λ -Productions are undesirable.

In a given CFG, we call a non-terminal N **nullable**

- if there is a production $N \rightarrow \lambda$, or
- there is a derivation that starts at N and lead to a λ .

$$N \rightarrow \dots\dots\dots \rightarrow \lambda$$

We can replace λ -production with appropriate non- λ productions.

Theorem

If L is CFL generated by a CFG having λ -productions, then there is a different CFG that has no λ -production and still generates either the whole language L (if L does not include λ) or else generate the language of all the words in L other than λ .

Proof by Construction (Replacement Rule):

1. Workout the Nullable Non-terminal
2. Delete all λ -Productions.
3. Add the following productions:

For every production of the $X \rightarrow \text{old string}$ Add new production of the form $X \rightarrow ..$, where right side will

account for every modification of the old string that can be formed by deleting all possible subsets of nullable non-Terminals, except that we do not allow $X \rightarrow \lambda$, to be formed if all the character in old string are nullable

- Delete all the Null productions and add new productions e.g.
- consider the productions of a certain CFG $X \rightarrow aNbNa$, $N \rightarrow \lambda$, delete the production $N \rightarrow \lambda$ and using the
- production $X \rightarrow aNbNa$, add the new productions $X \rightarrow aNba$, $X \rightarrow abNa$ and $X \rightarrow aba$
- Thus the new CFG will contain the productions

$$X \rightarrow aNba \mid abNa \mid aba \mid aNbNa$$
- Note: It is to be noted that $X \rightarrow aNbNa$ will still be included in the new CFG.

Example:

λ – production:

$$X \rightarrow \lambda$$

Nullable Variable:

$$Y \Rightarrow \dots \Rightarrow \lambda$$

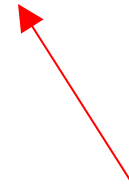
Example:

$$S \rightarrow aMb$$

$$M \rightarrow aMb$$

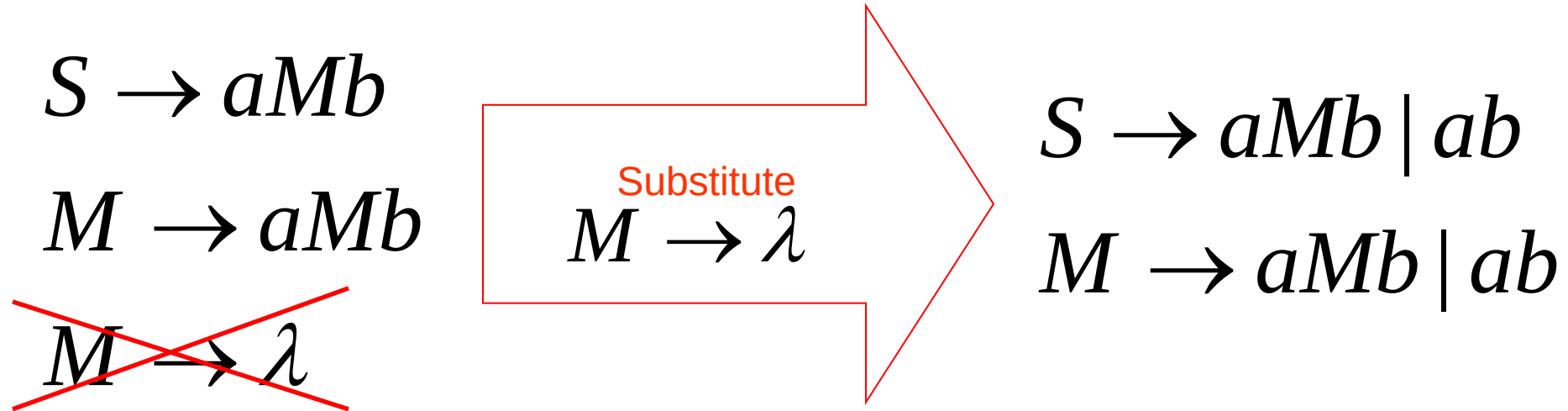
$$M \rightarrow \lambda$$

Nullable variable



λ – production

Removing λ – productions



After we remove all the λ – productions
all the nullable variables disappear
(except for the start variable)

Context Free Grammars Killing Unit Productions

- **Definition:** A production of the form
Nonterminal $\rightarrow$ one Nonterminal
is called a **unit production**.

The following theorem allows us to get rid of unit productions:

Theorem 24:

If there is a CFG for the language L that **has no λ -productions**, then there is also a CFG for L with no λ -productions and **no unit productions**.

Proof of Theorem

This is another proof by constructive algorithm

- **Algorithm:** For every pair of nonterminals A and B, if the CFG has a unit production $A \rightarrow B$, or if there is a chain

$$A \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow B$$

where $X_1, X_2, \dots$ are nonterminals, create new productions as follows:

- If the non-unit productions from B are

$$B \rightarrow s_1 \mid s_2 \mid \dots$$

where $s_1, s_2, \dots$ are strings, we create the productions

$$A \rightarrow s_1 \mid s_2 \mid \dots$$

Example

- Consider the CFG

$$S \rightarrow A \mid bb$$

$$A \rightarrow B \mid b$$

$$B \rightarrow S \mid a$$

- The non-unit productions are

$$S \rightarrow bb \quad A \rightarrow b \quad B \rightarrow a$$

- And unit productions are

$$S \rightarrow A$$

$$A \rightarrow B$$

$$B \rightarrow S$$

- Let's list all unit productions and their sequences and create new productions:

$S \rightarrow A$	gives	$S \rightarrow b$
$S \rightarrow A \rightarrow B$	gives	$S \rightarrow a$
$A \rightarrow B$	gives	$A \rightarrow a$
$A \rightarrow B \rightarrow S$	gives	$A \rightarrow bb$
$B \rightarrow S$	gives	$B \rightarrow bb$
$B \rightarrow S \rightarrow A$	gives	$B \rightarrow b$

- Eliminating all unit productions, the new CFG is

$S \rightarrow bb \mid b \mid a$
 $A \rightarrow b \mid a \mid bb$
 $B \rightarrow a \mid bb \mid b$

- This CFG generates a finite language since there are no nonterminals in any strings produced from S.

Unit Production:

$$X \rightarrow Y$$

(a single variable in both sides)

Example:

$$S \rightarrow aA$$

$$A \rightarrow a$$

$$A \rightarrow B$$

$$B \rightarrow A$$

Unit Productions

$$B \rightarrow bb$$

Removal of unit productions:

$$S \rightarrow aA$$

$$A \rightarrow a$$

~~$$A \rightarrow B$$~~

$$B \rightarrow A$$

$$B \rightarrow bb$$

Substitute

$$A \rightarrow B$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A \mid B$$

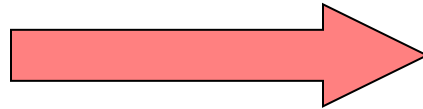
$$B \rightarrow bb$$

Remove repeated productions

$$S \rightarrow \textcircled{aA} \mid aB \mid \cancel{aA}$$

$$A \rightarrow a$$

$$B \rightarrow bb$$



Final grammar

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow bb$$

Useless Symbols

For a variable to be useful two conditions must be satisfied.

1. The variable must occur in a sentential form of the grammar
2. There must be a derivation of a terminal string from the variable.
 - A variable that occurs in a sentential form is said to be reachable from S.

A two part procedure is presented to eliminate useless symbols.

Algorithm to remove useless symbols

PART-I (from bottom to upward)

Identify variables that derive terminal strings

Remove non-terminals that do not derive terminal strings.

e.g. following grammar $G = S \rightarrow aS \mid A \mid C$

$$A \rightarrow a$$
$$B \rightarrow aa$$
$$C \rightarrow CB$$

Example:

$$S \rightarrow aS \mid A \mid C$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$C \rightarrow CB$$

We can identify variables that derive terminal strings.

$$\text{i.e. } A \rightarrow a \qquad B \rightarrow aa$$

$$\text{And } S \rightarrow A \rightarrow a \qquad \text{TERM} = \{S, A, B\}$$

But C does not derive terminal, thus C is useless

Example:

PART – II (from Top down)

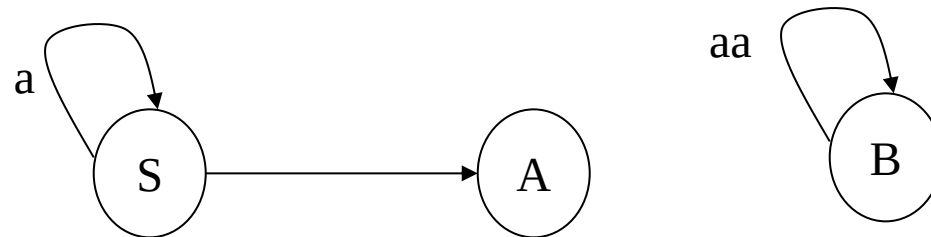
Rename the grammar

$$GT = S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

Now draw a graph and delete nodes not reachable from S.



As **B is unreachable** so delete it and the final grammar will be $G_V =$

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

$$S \rightarrow A$$

$$A \rightarrow aA$$

Useless Production

Some derivations never terminate...

$$S \Rightarrow A \Rightarrow aA \Rightarrow aaA \Rightarrow \dots \Rightarrow aa \dots aA \Rightarrow \dots$$

Another Grammar

$$S \rightarrow A$$

$$A \rightarrow aA$$

$$A \rightarrow \lambda$$

$$B \rightarrow bA$$

Useless Production

Not reachable from S

A production $A \rightarrow x$ is useless
if any of its variables is useless

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

Productions

Variables

$$S \rightarrow A$$

useless

useless

$$A \rightarrow aA$$

useless

useless

$$B \rightarrow C$$

useless

useless

$$C \rightarrow D$$

useless

Removing Useless Productions

Example Grammar:

$$S \rightarrow aS \mid A \mid C$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$C \rightarrow aCb$$

First: find all variables that can produce strings with only terminals

$$S \rightarrow aS \mid A \mid C$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$C \rightarrow aCb$$

Round 1: $\{A, B\}$

$$S \rightarrow A$$

Round 2: $\{A, B, S\}$

Keep only the variables
that produce terminal symbols: $\{A, B, S\}$
(the rest variables are useless)

$$S \rightarrow aS \mid A \mid \cancel{C}$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$\cancel{C \rightarrow aCb}$$



$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

Remove useless productions

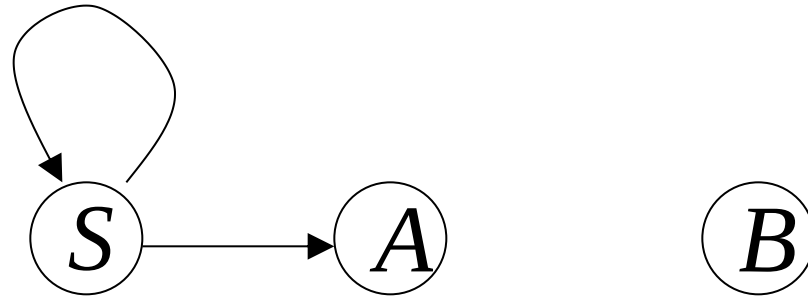
Second: Find all variables
reachable from S

Use a Dependency Graph

$S \rightarrow aS \mid A$

$A \rightarrow a$

$B \rightarrow aa$



not
reachable

Keep only the variables
reachable from S

(the rest variables are useless)

Final Grammar

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

~~$$B \rightarrow aa$$~~



$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

Remove useless productions

Example

- Consider following CFG

G: $S \rightarrow AC \mid BS \mid B$

$A \rightarrow aA \mid aF$

$B \rightarrow CF \mid b$

$C \rightarrow cC \mid D$

$D \rightarrow aD \mid BD \mid C$

$E \rightarrow aA \mid BSA$

$F \rightarrow bB \mid b$

A Two stage procedure

Algorithm 5.3.2

Construction of the Set of Variables That Derive Terminal Strings

input: context-free grammar $G = (V, \Sigma, P, S)$

1. $TERM := \{A \mid \text{there is a rule } A \rightarrow w \in P \text{ with } w \in \Sigma^*\}$
 2. repeat
 - 2.1. $PREV := TERM$
 - 2.2. for each variable $A \in V$ do
 - if there is an A rule $A \rightarrow w$ and $w \in (PREV \cup \Sigma)^*$ then
 - $TERM := TERM \cup \{A\}$
- until $PREV = TERM$
-

Satge-1 :

Construction of the set of Variables that derive terminal string

$S \rightarrow AC \mid BS \mid B$

$A \rightarrow aA \mid aF$

$B \rightarrow CF \mid b$

$C \rightarrow cC \mid D$

$D \rightarrow aD \mid BD \mid C$

$E \rightarrow aA \mid BSA$

$F \rightarrow bB \mid b$

New Grammar from
TERM will be

G_T :

$S \rightarrow BS \mid B$

$A \rightarrow aA \mid aF$

$B \rightarrow b$

$E \rightarrow aA \mid BSA$

$F \rightarrow bB \mid b$

Iteration	TERM	PREV
0	{B, F}	{ }
1	{B, F, A, S}	{B, F}
2	{B, F, A, S, E}	{B, F, A, S}
3	{B, F, A, S, E}	{B, F, A, S, E}

Algorithm 5.3.4

Construction of the Set of Reachable Variables

input: context-free grammar $G = (V, \Sigma, P, S)$

1. $REACH := \{S\}$
 2. $PREV := \emptyset$
 3. repeat
 - 3.1. $NEW := REACH - PREV$
 - 3.2. $PREV := REACH$
 - 3.3. for each variable $A \in NEW$ do
for each rule $A \rightarrow w$ do add all variables in w to $REACH$
- until $REACH = PREV$
-

Construction of the set of Reachable Variables

Original Grammar

$S \rightarrow AC \mid BS \mid B$

$A \rightarrow aA \mid aF$

$B \rightarrow CF \mid b$

$C \rightarrow cC \mid D$

$D \rightarrow aD \mid BD \mid C$

$E \rightarrow aA \mid BSA$

$F \rightarrow bB \mid b$

New Grammar from

TERM will be

G_T :

$S \rightarrow BS \mid B$

$A \rightarrow aA \mid aF$

$B \rightarrow b$

$E \rightarrow aA \mid BSA$

$F \rightarrow bB \mid b$

Final Grammar will be

G_F :

$S \rightarrow BS \mid B$

$B \rightarrow b$

Iteration	REACH	PREV	NEW
0	{S}	ϕ	{ }
1	{S, B}	{S}	{S}
2	{S, B}	{S, B}	{B}

Removing All

- **Step 0:** Adding a new start symbol by $S_0 \rightarrow S$ if the original start symbol 'S' is recursive.
- **Step 1:** Remove Nullable Variables
- **Step 2:** Remove Unit-Productions
- **Step 3:** Remove Useless Variables

Normal Forms for Context-free Grammars

CNF

Definition

If a CFG has only productions of the form

Non-Terminal $\rightarrow$ string of exactly two non-terminals
OR

Non-Terminal $\rightarrow$ one terminal

It is said to be in Chomsky Normal Form.

CNF

- It is to be noted that any CFG can be converted to be in CNF, if the null productions and unit productions are removed.
- Also if a CFG contains nullable productions as well, then the corresponding new production are also to be added.

CNF Theorem

- If L is a language generated by some CFG, then there exist another CFG that generate all the non-null words of L , all of whose productions are of one of the two basic forms:

Non-Terminal $\rightarrow$ string of only Non-terminals

Or

Non-Terminal $\rightarrow$ one Terminal

Theorem Proof

- The proof is a constructive algorithm by converting a given CFG in a CFG of the production of designed format.
- Let us suppose in given CFG the non-terminal are $S, X_1, X_2, X_3, \dots$.
Let us also assume $\{a, b\}$ are two terminals.

STEP-1

We now add two new non-terminals A and B and the productions $A \rightarrow a$ $B \rightarrow b$

STEP-2

now for every previous production involving terminals, we replace each 'a' with the non-terminal A and each 'b' with B.

e.g. $X_3 \rightarrow X_4aX_1SbbX_7a$

Becomes

$X_3 \rightarrow X_4AX_1SBBX_7A$

Which is a string of solid non-terminals

$X_6 \rightarrow abaab$

Becomes

$X_6 \rightarrow ABAAB$

Example

$$S \rightarrow X_1 \mid X_2 a X_2 \mid a S b \mid b$$

$$X_1 \rightarrow X_2 X_2 \mid b$$

$$X_2 \rightarrow a X_2 \mid a a X_2$$

$$S \rightarrow X_1 \mid X_2 A X_2 \mid A S B \mid B$$

$$X_1 \rightarrow X_2 X_2 \mid B$$

$$X_2 \rightarrow A X_2 \mid A A X_2$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Example

- Right sides of other production may contain terminals and non-terminals

e.g. $N \rightarrow aM \mid NaMb$

- We need to have string of solid non-terminal on right side

we introduce now products as $X \rightarrow a$ $Y \rightarrow b$

And apply them to all rules getting

$$N \rightarrow XM \mid NXMY$$

Theorem

For any context-free language L , the non- Λ words of L can be generated by a grammar in which all productions are in CNF.

Proof of theorem

- The proof will be by constructive algorithm.
- From Theorems 23 and 24, we know that there is a CFG for L (or for all L except Λ) that has no Λ -productions and no unit productions.
- Suppose further that we have made the CFG fit the form specified in Theorem 25.
- For productions of the form
Nonterminal $\rightarrow$ one terminal
we leave them alone.
- For each production of the form
Nonterminal $\rightarrow$ string of Nonterminals
we use the following expansion that involves new nonterminals $R_1, R_2, \dots$

Proof of Theorem

- For instance, the production $S \rightarrow X_1X_2X_3X_8$ should be replaced by $S \rightarrow X_1R_1$
where $R_1 \rightarrow X_2R_2$ and where $R_2 \rightarrow X_3X_8$.
- Thus, we convert a production with long string of non-terminals into a sequence of productions with exactly two non-terminals.
- If we need to convert another production, we introduce new R's with different subscripts.
- The new grammar generates the same language as the old grammar with the possible exception of the word Λ .

Example

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid aa \mid bb$$

$S \rightarrow ASA$	$\dashrightarrow$	$S \rightarrow AR_2$
$S \rightarrow BSB$	$\dashrightarrow$	$R_2 \rightarrow SA$
$S \rightarrow AA$		
$S \rightarrow BB$		$S \rightarrow BR_1$
		$R_1 \rightarrow SB$

$$S \rightarrow AA$$

$$S \rightarrow BB$$

$$S \rightarrow a$$

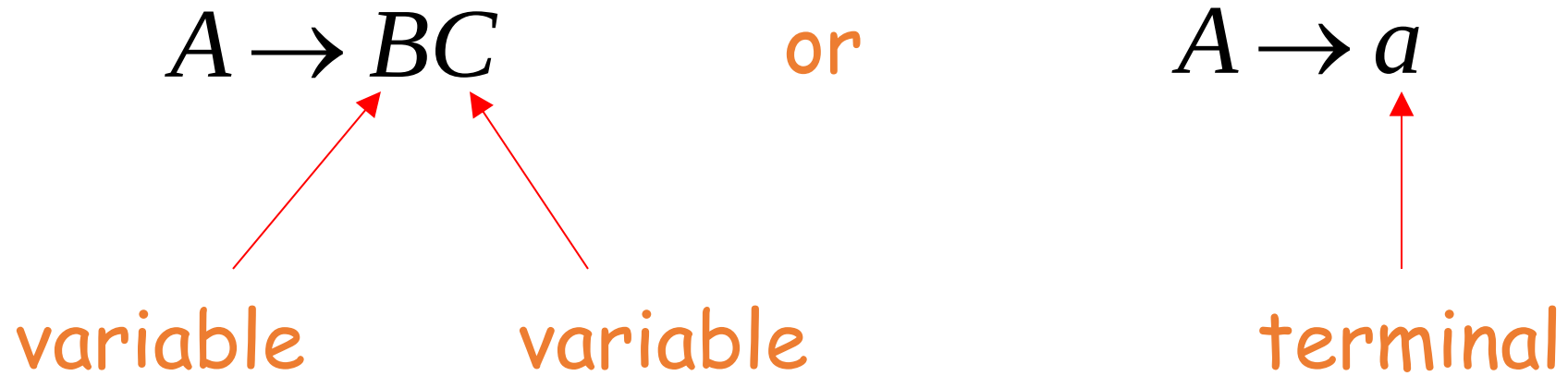
$$S \rightarrow b$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Chomsky Normal Form

Each production has form:



Examples:

$$S \rightarrow AS$$

$$S \rightarrow a$$

$$A \rightarrow SA$$

$$A \rightarrow b$$

Chomsky
Normal Form

$$S \rightarrow AS$$

$$S \rightarrow AAS$$

$$A \rightarrow SA$$

$$A \rightarrow aa$$

Not Chomsky
Normal Form

Conversion to Chomsky Normal Form

Example:

$$S \rightarrow ABa$$

$$A \rightarrow aab$$

$$B \rightarrow Ac$$

Not Chomsky
Normal Form

We will convert it to Chomsky Normal Form

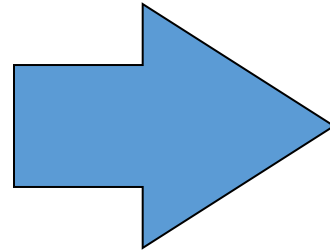
Introduce new variables for the terminals:

$$T_a, T_b, T_c$$

$$S \rightarrow ABa$$

$$A \rightarrow aab$$

$$B \rightarrow Ac$$



$$S \rightarrow ABT_a$$

$$A \rightarrow T_aT_aT_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

Introduce new intermediate variable V_1
to break first production:

$$S \rightarrow ABT_a$$

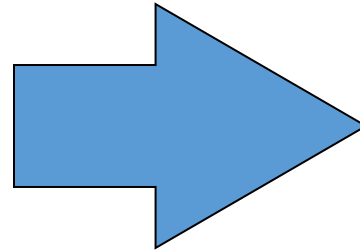
$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

Introduce intermediate variable: V_2

$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

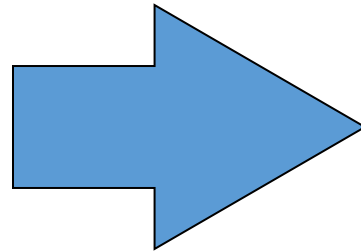
$$A \rightarrow T_a T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$



$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a V_2$$

$$V_2 \rightarrow T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

Final grammar in Chomsky Normal Form:

Initial grammar

$$S \rightarrow ABa$$

$$A \rightarrow aab$$

$$B \rightarrow Ac$$

$$S \rightarrow AV_1$$

$$V_1 \rightarrow BT_a$$

$$A \rightarrow T_a V_2$$

$$V_2 \rightarrow T_a T_b$$

$$B \rightarrow AT_c$$

$$T_a \rightarrow a$$

$$T_b \rightarrow b$$

$$T_c \rightarrow c$$

In general:

From any context-free grammar
(which doesn't produce ε)
not in Chomsky Normal Form

we can obtain:
an equivalent grammar
in Chomsky Normal Form